Experiment 'p_6_10_same_set_array' Results

June 4, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 1
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_6_10_same_set_array
Natural language statement: Write a method that checks whether two arrays have the same elements in some order, ignoring duplicates.
Method signature: p_6_10_same_set_array(first: array<int>, second: array<int>) returns (are_same_set: bool)

Ensures

- `are_same_set == (array_to_set(first[..]) == array_to_set(second[..]))`

Functional Code Given

```
function array_to_set(arr: seq<int>): set<int>
{
  if |arr| == 0 then {}
  else {arr[0]} + array_to_set(arr[1..])
}
```

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method that checks whether two arrays have the same elements in some order,
    ↪ ignoring duplicates.

The signature should be:

p_6_10_same_set_array(first: array<int>, second: array<int>) returns (are_same_set: bool)

The method should respect the following contract:

ensures are_same_set == (array_to_set(first[..]) == array_to_set(second[..]))

The contract uses the following dafny code:

function array_to_set(arr: seq<int>): set<int>
{
  if |arr| == 0 then {}
  else {arr[0]} + array_to_set(arr[1..])
}

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-Dn2STVyMhUNtLoMw5s4CUMeBYxtW4
Model: gpt-5.5-2026-04-23
Created at: 1780579893
Finish reason: stop
Usage: 434 tokens in, and 6186 tokens out

```
{
  var firstSet: set<int> := {};
  var i := first.Length;
  while i > 0
    invariant 0 <= i <= first.Length
    invariant firstSet == array_to_set(first[i..])
    decreases i
  {
    var k := i - 1;
    assert 0 <= k < first.Length;
```

```
    assert firstSet == array_to_set(first[k + 1..]);
    assert |first[k..]| > 0;
    assert first[k..][0] == first[k];
    assert first[k..][1..] == first[k + 1..];
    assert array_to_set(first[k..]) == {first[k]} + array_to_set(first[k + 1..]);
    firstSet := {first[k]} + firstSet;
    i := k;
    assert firstSet == array_to_set(first[i..]);
  }

  var secondSet: set<int> := {};
  var j := second.Length;
  while j > 0
    invariant 0 <= j <= second.Length
    invariant secondSet == array_to_set(second[j..])
    decreases j
  {
    var k := j - 1;
    assert 0 <= k < second.Length;
    assert secondSet == array_to_set(second[k + 1..]);
    assert |second[k..]| > 0;
    assert second[k..][0] == second[k];
    assert second[k..][1..] == second[k + 1..];
    assert array_to_set(second[k..]) == {second[k]} + array_to_set(second[k + 1..]);
    secondSet := {second[k]} + secondSet;
    j := k;
    assert secondSet == array_to_set(second[j..]);
  }

  assert first[0..] == first[..];
  assert second[0..] == second[..];
  assert firstSet == array_to_set(first[..]);
  assert secondSet == array_to_set(second[..]);
  are_same_set := firstSet == secondSet;
}
```

Final Program

```
function array_to_set(arr: seq<int>): set<int>
{
  if |arr| == 0 then {}
  else {arr[0]} + array_to_set(arr[1..])
}

method p_6_10_same_set_array(first: array<int>, second: array<int>) returns (are_same_set
    ↪ : bool)
      ensures are_same_set == (array_to_set(first[..]) == array_to_set(second[..]))
{
  var firstSet: set<int> := {};
  var i := first.Length;
```

```
  while i > 0
    invariant 0 <= i <= first.Length
    invariant firstSet == array_to_set(first[i..])
    decreases i
  {
    var k := i - 1;
    assert 0 <= k < first.Length;
    assert firstSet == array_to_set(first[k + 1..]);
    assert |first[k..]| > 0;
    assert first[k..][0] == first[k];
    assert first[k..][1..] == first[k + 1..];
    assert array_to_set(first[k..]) == {first[k]} + array_to_set(first[k + 1..]);
    firstSet := {first[k]} + firstSet;
    i := k;
    assert firstSet == array_to_set(first[i..]);
  }

  var secondSet: set<int> := {};
  var j := second.Length;
  while j > 0
    invariant 0 <= j <= second.Length
    invariant secondSet == array_to_set(second[j..])
    decreases j
  {
    var k := j - 1;
    assert 0 <= k < second.Length;
    assert secondSet == array_to_set(second[k + 1..]);
    assert |second[k..]| > 0;
    assert second[k..][0] == second[k];
    assert second[k..][1..] == second[k + 1..];
    assert array_to_set(second[k..]) == {second[k]} + array_to_set(second[k + 1..]);
    secondSet := {second[k]} + secondSet;
    j := k;
    assert secondSet == array_to_set(second[j..]);
  }

  assert first[0..] == first[..];
  assert second[0..] == second[..];
  assert firstSet == array_to_set(first[..]);
  assert secondSet == array_to_set(second[..]);
  are_same_set := firstSet == secondSet;
}
```

Total Token Usage

Input tokens: 434
Output tokens: 6186
Reasoning tokens: 5633
Sum of 'total tokens': 6620

Experiment Timings

Overall Experiment started at 1780579889298, ended at 1780580070710, lasting 181412ms (181.41 seconds)
Iteration #1 started at 1780579889339, ended at 1780580070705, lasting 181366ms (181.37 seconds)